LOCAL LLMs FOR AGENTIC QUALITY ENGINEERING

An Empirical Evaluation of Four Open-Weight Models for Multi-Agent Test Automation Pipelines

Srinidhi Sreevatsa
Associate Director & Lead QE Consultant
Independent Research

June 2026
Version 3.0

Abstract

As agentic AI transforms software quality engineering, organizations face a critical cost-versus-control tradeoff: cloud-hosted LLMs deliver superior instruction-following but incur significant per-token costs and raise data sovereignty concerns, while locally-deployed open-weight models promise zero marginal[1] inference cost but remain unproven for complex, multi-agent QE workflows.

This whitepaper presents an empirical evaluation of four open-weight models — Devstral Small 2 (24B), Qwen3.6-35B-A3B (35B MoE), Gemma 4 26B-A4B (26B MoE), and Ornith-1.0-9B (9B Dense) — against a production Claude Sonnet 4.6 baseline, across the six-agent pipeline of the Agentic QE Framework v2. The quality gap between local and cloud models is driven by three factors: instruction-following depth, context utilization quality, and run-to-run consistency — limitations that better hardware alone does not address. However, the experiment reveals that training methodology can partially compensate: Ornith-1.0-9B, a 9B model using only 5 GB VRAM, scored 74% on the framework's 9-dimension scorecard — outperforming two models 3x its size (Devstral at 57%, Gemma 4 at 54%) — by employing self-scaffolding reinforcement learning that optimizes instruction internalization. The experiment also demonstrates that instruction delivery architecture — skills-based progressive disclosure versus flat instruction files — is an independent variable affecting quality: the same model achieved 24/24 Explorer steps with skills-based instructions versus 5/24 with flat files. At the 16 GB VRAM tier, local models show promise for code generation agents but are not yet production-ready due to hallucination risks, run-to-run quality variance, and output formatting gaps. The financial case remains modest with the Builder accounting for approximately 10% of pipeline credits.

1. Introduction

1.1 The Cost Problem in Agentic QE

Multi-agent QE pipelines are token-intensive by design. A single medium-complexity scenario run through a six-agent pipeline — Enrichment, Explorer, Builder, Executor, Reviewer, and Healer — consumes a significant volume of API credits, with the Explorer and Executor stages accounting for the majority of token consumption. These costs are incurred during test suite generation and maintenance — not during test execution, which is fully deterministic. When an enterprise maintains hundreds of scenarios across its portfolio — generating new tests, updating suites after UI changes, and healing broken tests after deployments — the cumulative generation and maintenance cost becomes a material line item. This creates pressure to find alternatives that reduce per-token cost without degrading pipeline output quality.

[1]Marginal cost refers to the cost of each additional inference run. With local models, once the hardware is purchased, every subsequent run incurs zero additional cost — unlike cloud APIs where each run is billed per token.

1.2 The Promise of Local LLMs

Open-weight models have reached a critical capability threshold. Models like Devstral Small 2 achieve 68% on SWE-bench Verified, while Gemma 4 scores 78.5% on HumanEval — benchmarks that would have been state-of-the-art for commercial APIs just 18 months ago. More recently, Ornith-1.0-9B exceeded Devstral's SWE-bench score at 69.4% using a 9B dense architecture and roughly one-third the VRAM, signalling that parameter efficiency is improving as quickly as raw capability. More importantly, the hardware to run these models locally has become accessible: a single consumer GPU with 16 GB VRAM can now run 24–26B parameter models at quantized precision. Such hardware fits within the budget of a personal developer workstation or a purpose-provisioned pilot machine, though it exceeds the standard-issue configuration at most IT services organizations, where developer laptops typically ship without discrete GPUs.

The question is no longer whether local models can generate code, but whether they can generate code that meets the specific structural and quality conventions of an enterprise automation framework. Generic benchmarks measure capability in isolation; they do not measure compliance with the volume of framework-specific rules that must be simultaneously active during a single code generation pass. Even with progressive skill disclosure — where instruction content is loaded on demand rather than upfront — the Builder agent must hold tens of thousands of tokens of concurrent context spanning code generation rules, keyword mappings, and structural conventions. It is this simultaneous instruction density, not file size per se, that challenges local models.

1.3 Objectives

This study set out to answer three questions. First, can open-weight models running on consumer-grade hardware produce test automation artifacts that meet production quality standards? Second, which agent roles in a multi-agent QE pipeline are viable candidates for local offloading? Third, what hybrid cloud-local architecture delivers the best cost-to-quality ratio for enterprise deployment? A fourth question emerged during the experiment and is addressed in Section 5: how much of the observed quality gap is attributable to model size versus training methodology and instruction delivery architecture?

2. Experiment Design

2.1 Hardware Configuration

All experiments ran on a personal developer workstation configured as a mid-range engineering desktop. This configuration exceeds typical IT services company provisioning — where standard-issue laptops often lack discrete GPUs — but is representative of what an organization

might approve for a dedicated AI or QE pilot program, or what technology companies provision as standard engineering desktops.

Component	Specification
GPU	NVIDIA GeForce RTX 5060 Ti — 16 GB VRAM
CPU	AMD Ryzen 7 9700X — 8 cores, 3.80 GHz
RAM	64 GB DDR5 (4800 MT/s)
Storage	2.73 TB NVMe (459 GB used at experiment start)
OS	Windows 11 Pro, Version 25H2
NVIDIA Driver	591.86, CUDA 13.1

2.2 Models Under Evaluation

Models were selected based on three criteria: they must fit within 16 GB VRAM at Q4_K_M quantization[2], they must carry a permissive open-source license for commercial use (Apache 2.0 or MIT), and they must rank in the top tier of publicly available coding benchmarks. Several larger models were evaluated but rejected: Qwen3.6-27B (dense) could not run due to GGUF incompatibility with Ollama, DeepSeek V4-Flash (284B) and Devstral 2 (123B) require server-class GPUs, and Qwen3-Coder (480B) is data-center-only.

Three models formed the original cohort. A fourth, Ornith-1.0-9B, was added following its release on June 25, 2026, after initial testing was complete. Its inclusion proved consequential: it is the most parameter-efficient model tested, exceeding Devstral Small 2's SWE-bench score with a 9B dense architecture at roughly one-third the VRAM, and its results challenge several conclusions drawn from the original three-model cohort.

Model	Architecture	Params	VRAM (Q4_K_M)	Key Benchmark	License
Devstral Small 2	Dense	24B / 24B	~14 GB	SWE-bench: 68%	Apache 2.0
Qwen3.6-35B-A3B	MoE	35B / 3B active	~5–6 GB	SWE-bench: 49.5%	Apache 2.0
Gemma 4 26B-A4B	MoE	26B / 4B active	~8 GB active	HumanEval: 78.5%	Apache 2.0
Ornith-1.0-9B	Dense (post-trained on Qwen 3.5)	9B / 9B	~5–6 GB	SWE-bench: 69.4%	MIT

[2]Q4_K_M is a 4-bit quantization format from the llama.cpp ecosystem. It compresses model weights from 16-bit (FP16) to 4-bit precision, reducing model size by approximately 4x — for example, shrinking a 24B-parameter model from ~48 GB to ~14 GB — while retaining approximately 95–98% of the original model's output quality. The 'K' denotes an improved quantization method that assigns different bit-widths to layers based on sensitivity, and 'M' indicates the medium compression tier, balancing size reduction against quality preservation.

Model	Architecture	Params	VRAM (Q4_K_M)	Key Benchmark	License
Claude Sonnet 4.6 (baseline)	Cloud API	N/A	N/A	Production baseline	Commercial

Ornith-1.0-9B (DeepReinforce AI) warrants particular attention. Beyond its parameter efficiency, its distinguishing feature is a training methodology the developers term self-scaffolding[3] — jointly optimizing code generation and the orchestration framework that guides that generation, via reinforcement learning. Because the instruction-following gap was identified as the root cause of quality issues in the original three-model cohort, a model explicitly trained to internalize instruction sets represented a direct test of that diagnosis. It reports 43.1 on Terminal-Bench 2.1, offers a 128K context window, and ran at 69 tokens per second warm on the test hardware.

2.3 Tooling Stack

Models were served locally via Ollama v0.30.8 and integrated into the development environment through Cline v3.84, a VS Code coding agent extension with 62,000+ GitHub stars and 5M+ installs. Cline was selected after the initial choice, Roo Code (a Cline fork with multi-agent orchestrator mode), announced its final release and end-of-life during the experiment period, recommending users migrate to Cline or ZooCode (a community fork). This mid-experiment tooling disruption, while resolved without data loss, underscores the ecosystem maturity considerations discussed in Section 5.5. Cline was configured with a 65,536-token context window and 120-second request timeout. Playwright MCP was configured for browser automation. The Agentic QE Framework v2 provided the agent orchestration, instruction files, and quality assessment infrastructure.

Note on Cline's MCP Behavior with Ollama

Cline's Ollama provider displays a warning: "Does not support MCP and Focus Chain." Despite this warning, MCP tool calls did execute successfully during Explorer testing — the browser opened, Playwright navigation occurred, and accessibility snapshots were captured. The limitation manifested not as a connectivity failure but as extreme performance degradation (5–10 seconds per MCP round-trip) and incomplete output quality. This nuance is important: organizations evaluating Cline for MCP-dependent agents should test actual tool-call throughput rather than relying on the provider compatibility warning alone.

2.4 Evaluation Methodology

[3]Self-scaffolding, as described by DeepReinforce AI, is a reinforcement learning approach in which the model is optimized not only on the code it produces but also on the intermediate orchestration structure it builds to guide that production — effectively learning to construct its own working scaffold for a task. For instruction-dense workflows such as framework-compliant code generation, this targets the same capability the Architecture–Convention Gap measures.

All models were tested against the same scenario: a 23-step SauceDemo checkout-flow covering login, product selection, cart operations, checkout form entry, order review, and completion confirmation. The scenario spans 6 pages and includes 7 explicit VERIFY assertions. This scenario was chosen because it exercises the full range of Builder capabilities: page object generation, locator externalization, assertion translation, and multi-page navigation handling.

Output quality was assessed using the framework's 9-dimension Reviewer scorecard, with each dimension scored on a 1–5 scale. The verdict rules require an overall score of 80% or above, no individual dimension below 3, and Dimension 9 (Fidelity) at or above 4 to receive an APPROVED status. The same scorecard and rubric were applied to all models.

One methodological variable was introduced during the Ornith testing and proved significant enough to report as a finding in its own right. The original three models were tested exclusively with flat agent wrapper files (agents/cline/builder.md and equivalents) — a single instruction document per agent. Ornith-1.0-9B was tested under both flat wrappers and the framework's production skills-based agent architecture, which uses progressive skill disclosure to load instruction content on demand. Holding the model constant while varying only the instruction delivery method isolated that method as an independent variable, with results reported in Sections 4.5 and 5.7.

3. Agent Architecture

3.1 The Six-Agent Pipeline

The Agentic QE Framework v2 decomposes test automation into a sequential pipeline of specialized agents, each with a distinct responsibility and token profile. An Orchestrator coordinates the flow, and a Healer agent provides remediation when the Reviewer scorecard indicates quality failures.

Agent	Responsibility	MCP Required	Token Volume	Local Viable?
Enrichment	Convert NL/Swagger to structured scenario .md	No	Medium	Not yet tested
Explorer	Navigate live app via Playwright MCP, capture selectors	Yes	Highest	Partially (see 4.5)
Builder	Generate locator JSONs, page objects, and spec files	No	High	Yes (tested)
Executor	Run tests, fix timing and assertion issues	No	High	Possible

Agent	Responsibility	MCP Required	Token Volume	Local Viable?
Reviewer	9-dimension quality audit, produce scorecard	No	High	Not recommended
Healer	Diagnose and fix code quality issues from scorecard	No	Medium	Possible

3.2 The 9-Dimension Quality Scorecard

The Reviewer agent evaluates Builder output against nine quality dimensions. Each dimension carries a weight reflecting its impact on test reliability and maintainability. Dimension 9, Fidelity, acts as a hard gate: if the generated code does not faithfully implement all steps from the enriched scenario, the output cannot be approved regardless of scores on other dimensions.

#	Dimension	Weight	What It Measures
1	Locator Quality	High	Externalization, fallback count, selector accuracy
2	Wait Strategy	High	Presence of waitForLoadState, waitForURL, network idle
3	Test Architecture	Medium	POM pattern, test.step wrappers, tag annotations
4	Configuration	Medium	Framework config compliance (scored separately)
5	Code Quality	Low	TypeScript correctness, async/await, naming conventions
6	Maintainability	Medium	Data externalization, page-add extensibility pattern
7	Security	High	Credential handling via process.env, no hardcoded secrets
8	API Test Quality	Medium	API assertion patterns (N/A for pure web scenarios)
9	Fidelity	Critical	Step count match, VERIFY translation, builder report accuracy

3.3 Token Efficiency by Design: Scripts for Evidence, LLMs for Judgment

The framework embeds a core architectural principle: deterministic tasks that can be computed without reasoning are handled by Node.js scripts at zero token cost. The LLM is invoked only for tasks requiring judgment, classification, or generation. This principle, termed “scripts for evidence, LLMs for judgment,” is not an optimization applied after the fact — it is a structural design decision that shapes every agent’s workflow.

Nine deterministic scripts form the framework’s zero-token infrastructure:

Script	Purpose	What It Replaces (If Done by LLM)
review-precheck.js	Collects file manifests, counts selectors, checks for raw selector leaks, validates step counts, gathers assertion evidence	Reviewer reading every generated file line-by-line
test-results-parser.js	Parses Playwright JSON output into structured failure data	LLM parsing raw JSON (often 10–50 KB per run)
failure-classifier.js	Categorizes test failures deterministically	LLM reasoning about each failure individually
explorer-post-check.js	Verifies Explorer output completeness mechanically	LLM re-reading enriched.md and cross-checking
builder-incremental.js	Diffs scenario changes, identifies what Builder must regenerate	LLM reading old and new scenario files (2x context)
scenario-diff.js	Section-aware diff with change classification	LLM comparing two files token-by-token
cleanup-annotations.js	Strips Builder markers from generated code	LLM reading generated files to find and remove markers
metrics-collector.js	Aggregates pipeline observability data	LLM gathering timing and count data from reports
rehash-skills.js	Updates skill content hashes for drift detection	LLM reading and checksumming skill files

This architecture has direct implications for the local LLM evaluation. When a local model serves as the Builder, the scripts handle all mechanical pre- and post-processing, limiting the model's responsibility to the reasoning-intensive generation step. The token budget for a local Builder run is therefore smaller than it would be in a naive architecture where the LLM handles everything — and smaller token budgets mean the model's context window is less pressured, which partially mitigates the instruction-following degradation observed at high context utilization.

4. Experiment Results

4.1 Experiment 1: Builder Agent — Devstral Small 2 (24B)

4.1.1 Performance Profile

Devstral Small 2 ran entirely on GPU, confirmed at 100% utilization. The model evaluated at approximately 20–30 tokens per second, completing the full Builder run in approximately 40–45 minutes of wall-clock time (including instruction file reading, Cline overhead, and code generation) — roughly eight to nine times the time required by Claude Sonnet 4.6 on the same scenario. The model required an explicit, multi-line prompt; the short-form "Run Builder" instruction that works with Claude did not trigger correct behavior.

4.1.2 Output Assessment

Devstral generated the correct file structure: 13 files comprising 6 page objects, 6 locator JSON files, and 1 spec file. All 23 scenario steps were present in correct order, and all 7 VERIFY assertions were correctly translated to Playwright expect() calls. The model demonstrated clear understanding of the framework's architectural patterns: BasePage inheritance, LocatorLoader integration, and locator externalization with zero raw selectors in page objects.

4.1.3 Scorecard Results

Dimension	Score	Key Finding
Dim 1 · Locator Quality	3/5	Good externalization but login primaries used wrong selector format (testid= instead of id=)
Dim 2 · Wait Strategy	2/5	No wait strategy at all — zero waitForLoadState or waitForURL calls
Dim 3 · Test Architecture	3/5	Clean POM but test.step() wrappers missing; contradictory storageState line
Dim 5 · Code Quality	4/5	Clean TypeScript, correct async/await, meaningful method names
Dim 6 · Maintainability	3/5	Good page-add pattern but scenario data hardcoded in spec
Dim 7 · Security	2/5	Credentials hardcoded directly in spec instead of process.env
Dim 9 · Fidelity	3/5	All steps present but test.step() wrappers missing; builder report fabricated metrics

Overall: 20/35 (57%) — NEEDS FIXES
Two dimensions scored below the minimum threshold of 3 (Wait Strategy and Security). Fidelity scored 3, below the hard gate of 4 required for approval. Three runtime errors required manual correction before the generated tests could execute.

4.1.4 Runtime Errors

Three errors required post-generation fixes. First, Devstral hallucinated a storageState configuration line referencing a non-existent auth file, contradicting the explicit login steps in the scenario. Second, the login page's primary locators used the testid= format (resolving to data-testid), but SauceDemo uses id= attributes — the correct selectors were present in fallback positions but unreachable by the framework's loc.get() method, which by design returns only the primary selector. The framework provides a separate getWithFallback() method for runtime resilience, but the Builder is expected to set correct primaries — making this a model quality issue, not a framework design gap. Third, the spec asserted a wrong subtotal amount ($49.99 instead of $39.98), though this was traced to a pre-existing error in the enriched.md input rather than a Builder fault. This error is included not as a Builder fault — the model correctly reproduced the value present in its input — but as evidence that upstream errors propagate

faithfully through the pipeline. Input validation between pipeline stages would catch such discrepancies before they reach the Builder.

A fourth issue, while not a runtime error, affected locator quality across both Builder experiments. The enriched.md input file contained simplified LOCATOR annotation comments rather than the full ELEMENT annotation format specified in the Builder's instruction files. The full format includes a JSON block with tag, id, testId, text, and cssPath fields that the Builder uses to generate the fingerprint section of locator JSON files. With only LOCATOR tags available, models were forced to infer selector strategies from limited information. Gemma 4 detected this discrepancy and paused for clarification; Devstral proceeded without flagging it and generated incorrect primary selectors for the login page as a result.

For comparison, Claude Sonnet 4.6 handles format variations more gracefully through contextual inference — it would typically generate reasonable locators from LOCATOR tags without pausing, using its broader understanding of Playwright selector conventions to compensate for the missing annotation metadata. This contrast illustrates a spectrum of model responses to ambiguous input: Claude infers silently and correctly, Gemma 4 detects the ambiguity and requests clarification, and Devstral proceeds silently but incorrectly.

4.2 Experiment 2: Explorer Agent — Qwen3.6-35B-A3B

4.2.1 Performance Profile

Qwen3.6-35B-A3B was tested as the Explorer agent, the most token-intensive role requiring live browser interaction via Playwright MCP. Despite its MoE architecture requiring only 3B active parameters and fitting comfortably in VRAM, the model ran 10–20x slower than Claude Sonnet. Each MCP tool-calling round-trip took 5–10 seconds, and the total run extended over several hours.

4.2.2 Results

The model completed only 8 of 23 steps, stopping at the inventory page. The cart, checkout information, checkout overview, and checkout completion pages were listed as "Expected but not yet captured." Steps were duplicated with CHANGE/WALK annotations, and the output used an incorrect enriched.md format — a separate "ELEMENT Annotations" section at the bottom instead of the required inline annotation format.

Verdict: Not Viable Under Flat Instruction Wrappers

The Explorer agent's combination of MCP tool-calling, live browser interaction, and format-sensitive output generation proved unsuitable for Qwen3.6-35B-A3B under flat instruction wrappers. On the evidence of this experiment alone, the conclusion drawn was that Explorer must remain on cloud inference. Section 4.5 revises that conclusion: a later model under a different instruction architecture completed the full navigation locally, isolating instruction delivery — not the Explorer role itself — as the binding constraint.

Qwen3.6-35B-A3B was selected for the Explorer role based on its 2x advantage on the MCPMark benchmark (37% vs 18% for the nearest competitor), which specifically measures MCP tool integration quality. Devstral Small 2 and Gemma 4 were not tested as Explorer agents because both have lower MCP benchmark scores than Qwen. Given that the model with the strongest MCP tool-calling profile failed to complete a 23-step exploration in acceptable time, testing models with weaker MCP capabilities was judged unlikely to yield a different outcome — a judgment that held until the instruction delivery method itself was varied.

4.3 Experiment 3: Builder Agent — Gemma 4 26B-A4B

4.3.1 Performance Profile

Gemma 4 ran at 64–76 tokens per second at warm state, 2–3x faster than Devstral Small 2 on the same GPU, while consuming only ~8 GB of active VRAM versus Devstral's ~14 GB. However, the model spent approximately 40 minutes reading and re-reading instruction files before beginning code generation. Notably, Gemma 4 asked a valid clarifying question about the LOCATOR versus ELEMENT annotation format — a distinction that Devstral did not detect. This is a significant finding: Gemma 4 demonstrated superior instruction comprehension by identifying an inconsistency between the input data and the documented specification, yet its overall output score was marginally lower than Devstral's. Better comprehension of instructions does not automatically translate to better compliance with those instructions during generation — a nuance that synthetic benchmarks, which measure generation quality in isolation, cannot capture.

4.3.2 Scorecard Results

Dimension	Score	Key Finding
Dim 1 · Locator Quality	4/5	Login primaries CORRECT (id=user-name); full schema with fingerprint field
Dim 2 · Wait Strategy	2/5	No waits — identical gap to Devstral
Dim 3 · Test Architecture	3/5	Clean POM, no storageState hallucination, but broken async expect pattern
Dim 5 · Code Quality	3/5	Clean TypeScript but broken assertion: await expect(promise).toBe()
Dim 6 · Maintainability	3/5	Same as Devstral — hardcoded scenario data
Dim 7 · Security	2/5	Credentials hardcoded — identical to Devstral
Dim 9 · Fidelity	2/5	Wrong postal code, folder typo propagated, 23 steps collapsed to 8

Overall: 19/35 (54%) — NEEDS FIXES

Score marginally below Devstral despite stronger locator quality and faster inference. The lower Fidelity score (2 vs 3) was driven by step collapsing and data errors. Like Devstral, Wait Strategy and Security fell below threshold.

A notable output integrity issue emerged during the Gemma 4 run: the model misspelled the folder parameter as ‘sauceddemo’ (double ‘d’) instead of ‘saucedemo,’ and this single typo propagated to three locations — the test spec output folder (output/tests/web/sauceddemo/), the reports folder (output/reports/sauceddemo/), and all file path references in the builder report. The model also created the correctly-named folder, resulting in duplicate output directories with files split across both. This error was not observed in the Devstral run. It illustrates a propagation risk specific to local models: a single early-stage mistake in path construction cascades through every subsequent file operation without self-correction — a behavior that Claude Sonnet 4.6, with its stronger self-consistency, typically avoids.

4.4 Experiment 4: Builder Agent — Ornith-1.0-9B (9B Dense)

Ornith-1.0-9B was tested as a Builder agent across two runs that differed in both input quality and instruction delivery method. The two runs produced markedly different scores — 74% and 60% — and the divergence is itself among the more instructive results in this study.

4.4.1 Run 1: Original Enriched Input, Flat Wrappers

The first run used the same conditions as the Devstral and Gemma 4 tests: the original enriched file (with LOCATOR tags rather than full ELEMENT annotations), flat agent wrapper files, and an explicit multi-line prompt. The model completed in approximately 10 minutes — four times faster than Devstral (40–45 minutes) and Gemma 4 (40+ minutes) on the identical task — and produced the full expected file set: 6 page objects, 6 locator JSONs, 1 spec, plus builder report and metrics JSON.

Dimension	Score	Key Finding
Dim 1 · Locator Quality	3/5	Full schema with notes field; primaries plausible but hallucinated (label=Username, h1) — will not match SauceDemo’s actual id=user-name
Dim 2 · Wait Strategy	4/5	waitFor({ state: ‘visible’, timeout: 30000 }) in verification methods — major improvement over Devstral (2/5) and Gemma 4 (2/5)
Dim 3 · Test Architecture	4/5	test.step() on every step (23 wrappers), tags @smoke @P0, process.env for all data including checkout fields
Dim 5 · Code Quality	3/5	Bug: this.loc.get() inside arrow function loses context; deprecated page.click(); duplicate console.log
Dim 6 · Maintainability	4/5	All test data externalized; notes field in locator JSONs
Dim 7 · Security	5/5	Zero hardcoded credentials — everything via process.env (TEST_USERNAME, TEST_PASSWORD, CHECKOUT_FIRST_NAME, CHECKOUT_ZIP, BASE_URL)
Dim 9 · Fidelity	3/5	All 23 steps present; added phantom Step 24 (reset to login — not in scenario); subtotal checks label only, not amount

Overall: 26/35 (74%) — Highest Local Score Recorded
No dimension fell below 3, clearing a threshold that both larger models failed. Security scored a perfect 5/5 — the only local model to fully externalize credentials. Fidelity at 3/5 remains below the hard gate of 4 required for APPROVED status, and the hallucinated locator primaries mean the generated tests would still fail at runtime without correction.

4.4.2 Convention Compliance Across Builder Models

The most striking aspect of the Ornith result is not its aggregate score but its distribution. Ornith closed nearly every convention gap that both larger models missed — the precise category of failure identified in Section 5.1 as the root cause of the local-cloud quality gap.

Convention	Devstral 24B	Gemma 4 26B	Ornith 9B
test.step() wrappers	Missing	Missing	Present on all 23 steps
Tags (@smoke, @P0)	Missing	Missing	Present in describe AND test options
process.env for credentials	Hardcoded	Hardcoded	process.env throughout
process.env for test data	Hardcoded	Hardcoded	process.env for checkout fields
Wait strategy	None	None	waitFor({ state: ‘visible’, timeout: 30000 })
storageState hallucination	Yes (caused error)	None	None
Correct folder name	saucedemo	sauceddemo (typo)	saucedemo
Locator fingerprint field	Missing	Present	Present + notes field
BasePage + loc.get()	Yes	Yes	Yes

Ornith-1.0-9B fixed nearly every convention gap that both larger models missed. The self-scaffolding training methodology — where the model learns to internalize instruction sets and build an internal framework from them — directly addresses the instruction-following gap identified as the root cause of quality issues in the original three-model experiment. This result is analysed further in Section 5.6.

4.4.3 Run 2: Explorer-Enriched Input, Skills-Based Architecture

The second run supplied what should have been superior input: the enriched file produced by the Ornith Explorer run described in Section 4.5, containing selectors captured from the live DOM rather than inferred. It also used the framework’s skills-based agent architecture rather than flat wrappers. Both changes were expected to improve output. The score fell instead, from 26/35 (74%) to 21/35 (60%).

The output shrank as well: only 2 page objects (LoginPage, InventoryPage) instead of 6, four legitimate locator JSON files plus one hallucinated file, a spec with 24 test.step() blocks, and a test data JSON with the correct $39.98 amount. Six distinct defects account for the regression.

1. God Object (critical POM violation). InventoryPage.ts contains 18+ methods spanning 6 pages — inventory browsing, cart verification, checkout form filling, order overview, confirmation, and navigation back home. The framework requires one page object per page; this consolidation makes the code unmaintainable.

2. Hallucinated locator file (severe — context contamination). filter-configuration-modal.locators.json was generated with locators for date range filters, BOL numbers, hauler checkboxes, invoice numbers, and truck numbers — none of which exist in SauceDemo. The content appears to originate from a logistics or BOL tracking application in the model's training data, mixed into output for an entirely unrelated application.

3. LocatorLoader API inconsistency. LoginPage uses this.loc.getLocator('key') while InventoryPage uses this.page.locator(this.loc.get('key')) — two different API patterns in the same codebase, one of which will fail at runtime.

4. Step 13 wrong method. The spec calls inventoryPage.clickContinue() for the Checkout button click. clickContinue() triggers the Continue button on the checkout info page, not the Checkout button on the cart page.

5. CartPage.ts dead code. CartPage.ts was created with proper methods but the spec never imports or uses it — all cart operations route through the InventoryPage god object.

6. Ephemeral selector references. role=generic[ref=e124] for cartBadge and role=generic[ref=e234] for checkoutOverviewHeading — these ref=eNNN values are session-specific accessibility tree references that change on every page load and will fail on the next run.

Dimension	Run 1 (old enriched, flat)	Run 2 (Explorer enriched, skills)
Dim 1 · Locator Quality	3/5	3/5
Dim 2 · Wait Strategy	4/5	4/5
Dim 3 · Test Architecture	4/5	3/5 (god object)
Dim 5 · Code Quality	3/5	2/5 (API inconsistency, wrong method)
Dim 6 · Maintainability	4/5	2/5 (god object)
Dim 7 · Security	5/5	4/5 (some hardcoded data)
Dim 9 · Fidelity	3/5	3/5
Total	**26/35 (74%)**	**21/35 (60%)**

Key Insight: Better Input Did Not Produce Better Output
Explorer-verified selectors and a richer instruction architecture together produced a worse result. The model's structural decisions degraded as context complexity rose — consolidating six page objects into one, hallucinating a locator file from an unrelated application, and mixing two incompatible LocatorLoader APIs. The simpler first run scored higher because the model had less context to process and made fewer structural errors. This run-to-run variance of 14 percentage points on the same model is, for production purposes, as significant a finding as the peak score itself.

4.5 Experiment 5: Explorer Agent — Ornith-1.0-9B

Ornith-1.0-9B was also tested in the Explorer role, against the same SauceDemo checkout-flow scenario and the same Playwright MCP configuration used for Qwen3.6-35B-A3B. As with the Builder tests, two runs were performed — and here the instruction delivery method was the only variable that changed between them.

4.5.1 Run 1: Flat Wrappers — 5/23 Steps

Using flat agent wrapper instructions, the model completed steps 1 through 5 (navigate, login, verify inventory) and then stopped. Its stated reason was that SauceDemo uses Shadow DOM and Web Components for product cards, rendering the "Add to Cart" buttons inaccessible to the automation layer. This is factually false: SauceDemo uses plain HTML div elements with standard CSS classes and data-test attributes. The model also hallucinated the cart URL as view-cart.html, where the actual path is cart.html.

This failure differs in kind from Qwen3.6-35B-A3B's. Qwen was slow and incomplete but did not fabricate technical explanations — it simply ran out of runway. Ornith invented a plausible-sounding technical barrier and documented it confidently, with detailed supporting observations. An engineer reading that output without independently inspecting the DOM would reasonably conclude the application was untestable through this route. This behaviour is examined as a distinct risk category in Section 5.8.

4.5.2 Run 2: Skills-Based Architecture — 24/24 Steps

The second run changed one thing: instructions were delivered through the framework's production skills-based agent architecture with progressive skill disclosure, rather than flat wrapper files. The model, the scenario, the hardware, and the MCP configuration were all identical. Ornith-1.0-9B completed all 24 scenario steps in approximately 10 minutes — the first local model in this study to complete full Explorer navigation.

What it got right. All 24 steps navigated successfully — login, add to cart, checkout form, verification, and return home. Twenty-one ELEMENT annotations were captured with the correct JSON schema (key, type, primary, fallbacks, fingerprint). Selectors were captured from the actual live DOM via browser_snapshot rather than inferred. The item total correctly showed

$39.98, the real value. The App Behavior section documented SPA async rendering with specific timing observations. Critically, there were zero hallucinations — no fabricated Shadow DOM, no invented URLs.

What it got wrong — formatting, not capability. ELEMENT annotations were clustered at the bottom of the file instead of placed inline below each step. LEDGER markers were missing or misplaced, so the explorer-report-gen.js script could not parse them — the generated report showed 0/24 steps verified and 0 elements captured despite the underlying work being complete and correct. The enriched file was saved to the wrong location (output/scenarios/web/ instead of scenarios/web/saucedemo/). The report lacked the detail level Claude Sonnet 4.6 produces.

> **Critical Insight: Instruction Delivery Method Is an Independent Variable**
> The difference between 5/23 with a fabricated technical excuse and 24/24 with zero hallucinations was not a model change, a hardware change, or a prompt-quality change. It was the same model on the same hardware against the same scenario, with instructions delivered through progressive skill disclosure rather than a flat instruction file. Instruction architecture — how rules are structured and surfaced to the model — is as consequential as model capability for local LLM performance in agentic workflows.

4.5.3 Explorer Results Across All Tests

Model	Instruction Method	Steps	Time	Hallucinations
Qwen3.6-35B-A3B	Flat wrappers	8/23	Hours	Wrong format, duplicated steps
Ornith-1.0-9B	Flat wrappers	5/23	Minutes	Shadow DOM fabrication
Ornith-1.0-9B	Skills-based	24/24	~10 min	None
Claude Sonnet 4.6	Production config	24/24	~5 min	None

The practical consequence for the recommended architecture is a split verdict. The Explorer's navigation capability — driving a live browser through 24 steps via MCP, capturing real DOM selectors, and doing so in a time frame comparable to cloud inference — is now demonstrably viable locally. The Explorer's reporting compliance — inline annotation placement, LEDGER marker emission, and correct file placement, all of which downstream framework scripts depend on — is not. Section 6.2 revises the architecture recommendation accordingly.

4.6 Head-to-Head: Four-Model Comparison

The following table consolidates results across all four local models. Blank entries reflect roles a model was not tested in — Qwen3.6-35B-A3B was selected for MCP tool-calling strength rather than code generation, and was not run as a Builder.

Metric	Devstral Small 2 (24B)	Qwen3.6-35B-A3B (35B MoE)	Gemma 4 26B-A4B (26B MoE)	Ornith-1.0-9B (9B Dense)
VRAM usage	~14 GB	~5–6 GB	~8 GB active	~5–6 GB
Inference speed	20–30 t/s	50–70 t/s	64–76 t/s	69 t/s
Builder time	40–45 min	Not tested as Builder	40+ min	~10 min
Builder score	20/35 (57%)	Not tested as Builder	19/35 (54%)	26/35 (74%)
Explorer result	Not tested	8/23 steps (hours)	Not tested	5/23 (flat) → 24/24 (skills-based)
test.step() wrappers	Missing	N/A	Missing	Present
process.env credentials	Hardcoded	N/A	Hardcoded	process.env
Wait strategy	None	N/A	None	waitFor with timeout
Tags (@smoke, @P0)	Missing	N/A	Missing	Present
Hallucination risk	Low (wrong selectors)	Low (wrong format)	Low (wrong postal code, typo)	HIGH (Shadow DOM fabrication, filter-config file)
Key strength	Step fidelity	MCP tool-calling speed	Locator schema compliance	Convention following
Key weakness	Convention gaps	Explorer incomplete	Assertion bugs, folder typo	Hallucination risk, structural decisions under complex input

Among the original three models, neither Devstral nor Gemma 4 achieved a clear overall advantage. Gemma 4 excels on inference efficiency (2–3x faster, nearly half the VRAM) and locator accuracy, while Devstral maintains better step-level fidelity and assertion correctness. Qwen3.6-35B-A3B was not tested as a Builder agent because its 49.5% SWE-bench score is substantially below both Devstral (68%) and Gemma 4 (78.5% HumanEval); with only 3B active parameters, it was selected for its MCP tool-calling strength (MCPMark 37%), not for code generation.

Ornith-1.0-9B changes the shape of the comparison rather than simply topping it. It leads on convention following, speed, and VRAM efficiency, and it is the only model to complete Explorer navigation. It also carries the highest hallucination risk of any model tested and the widest run-to-run variance. The correct reading is not that Ornith is the best local model for this framework, but that it is the first to demonstrate that the convention gap is closable at small parameter counts — while introducing a failure mode that is harder to detect than any observed in the larger models.

4.7 Inference Speed Comparison

Token generation speed directly impacts developer experience and pipeline throughput. All benchmarks were measured on the same RTX 5060 Ti GPU using Ollama's verbose output.

Model	Eval Rate (Warm)	Builder Run Time	Cold Start Load	vs Claude Sonnet 4.6
Devstral Small 2 (24B dense)	20–30 tokens/sec	~40–45 minutes	~15 seconds	~8–9x slower
Qwen3.6-35B-A3B (3B active)	50–70 tokens/sec	Hours (incomplete)	~5 seconds	10–20x slower (MCP loop)
Gemma 4 26B-A4B (4B active)	64–76 tokens/sec	~40 min (incl. re-reads)	~40 seconds (first run)	~4x slower (incl. reading)
Ornith-1.0-9B (9B dense)	69 tokens/sec	~10 minutes	~10 seconds	~2x slower
Claude Sonnet 4.6 (cloud)	N/A (API latency)	~5 minutes	N/A	Baseline

A critical observation is the gap between raw token throughput and effective task completion time. Among the original three models, eval rates of 20–76 tokens per second all yielded 40+ minute Builder runs against Claude Sonnet 4.6's approximately 5 minutes. That disparity was driven by instruction file processing overhead: models repeatedly re-read large instruction files rather than processing them in a single pass, consuming significant wall-clock time on context ingestion before code generation began.

Ornith-1.0-9B breaks this pattern in a way that reinforces the diagnosis. Its eval rate (69 t/s) sits between Gemma 4's and Devstral's, yet it completed the same task in a quarter of the time. The difference is not generation speed but ingestion behaviour — Ornith did not exhibit the repeated instruction re-reading cycle. A model trained to internalize instruction structure spends less time re-parsing it, which converts directly into wall-clock savings independent of tokens per second.

5. Cross-Model Analysis

5.1 The Architecture–Convention Gap

The most consistent finding across the original three models is what we term the "Architecture–Convention Gap." Every local model correctly implemented the framework's structural patterns: BasePage inheritance, LocatorLoader integration via this.loc.get(), zero raw selectors in page objects, correct file structure across page objects and locator JSONs, and accurate VERIFY-to-expect() assertion translation. These are the patterns visible in the framework's code structure — patterns a model can infer from reading the existing codebase.

What those models missed were the framework's conventions: explicit rules documented in instruction files rather than demonstrated in code structure. These include wrapping every step

in test.step() blocks, using process.env for credential handling, implementing wait strategies (waitForLoadState, waitForURL) after navigation events, externalizing test data to JSON files, and applying tag annotations (@smoke, @P0). These rules are distributed across multiple instruction documents that collectively represent tens of thousands of tokens of concurrent context. The instruction set is comprehensive by design — enterprise QE demands detailed conventions for locator patterns, assertion strategies, wait handling, security practices, and output structure. The challenge is not the instruction volume itself, which is justified by the framework's quality requirements, but the model's inability to sustain attention across the full rule set during generation.

Ornith-1.0-9B qualifies this finding without overturning it. It closed the convention gap almost entirely — test.step() wrappers on all 23 steps, tags in both describe and test options, process.env throughout, and an explicit wait strategy — while running at a third the parameter count of the models that missed those same rules. The gap is therefore not an inherent property of small models operating under dense instruction sets. It is a property of how a model was trained to process instruction sets, and separately, of how those instructions are delivered. Both are addressed below.

5.2 Common Failures

Six failures appeared consistently across the original three local models. No model generated test.step() wrappers despite the framework explicitly requiring them. No model implemented any wait strategy. All models hardcoded credentials instead of reading from environment variables. All models hardcoded test data instead of externalizing it. No model applied test tags. And no model produced structured builder report metrics with step count verification or self-audit data.

Ornith-1.0-9B resolved the first five of these six in its stronger run, leaving builder report metrics as the only failure common to all four models. In exchange, it introduced two failure modes the earlier models did not exhibit: confident fabrication of technical content (Section 5.8) and structural degradation under increased context complexity (Section 4.4.3). The set of common failures has therefore narrowed rather than disappeared, and the residual failures have become harder to detect through inspection.

5.3 Why Cloud Models Outperform

Claude Sonnet 4.6 typically scores 32–38 out of 40 (80–95%) on the same assessment. The gap is attributable to three factors. First, superior instruction-following from RLHF[4] training — Claude demonstrates significantly better compliance with explicit textual rules, even when those rules span large documents. Second, better context utilization across its 200K-token window, allowing it to hold the full instruction set in working memory while generating output. Third,

[4]Reinforcement Learning from Human Feedback (RLHF) is a training technique where human evaluators rate model outputs, and the model is fine-tuned to prefer responses that humans rank higher. This process is particularly effective at teaching models to follow complex instructions precisely and consistently.

higher consistency: the same correct output is produced approximately 9 out of 10 runs, compared to local models where each run may miss different conventions.

The Ornith results allow these three factors to be separated more precisely than the original cohort permitted. On the first factor — instruction-following — a 9B model narrowed the gap substantially, indicating that this component responds to training methodology rather than scale. The second and third factors proved more stubborn. Ornith's Run 2 regression under richer context is a context utilization failure, and its 14-point spread between runs is a consistency failure. Instruction-following, in other words, is the most tractable of the three, and the two that remain are precisely the ones that matter most for unattended production use.

5.4 The Fabrication Risk

Devstral's builder report claimed "23 test.step() blocks" when no such blocks existed in the generated code. This fabricated metric represents a significant risk for automated pipelines: if downstream agents or dashboards consume builder reports without independent verification, fabricated claims could mask quality gaps. The framework's design principle of "scripts for evidence, LLMs for judgment" exists precisely to guard against this, but the finding reinforces that local model outputs require the same Reviewer scrutiny as cloud model outputs. Section 5.8 documents a more severe variant of this risk observed in Ornith-1.0-9B.

5.5 Operational Findings

Three operational findings emerged during the experiment that have practical implications for teams adopting local LLMs for agentic workflows.

Prompt complexity requirements. Claude Sonnet 4.6 reliably executes the Builder from a one-line command: "@QE Builder scenario=checkout-flow type=web folder=saucedemo." All local model tests required explicit multi-line prompts that listed every instruction file to read, specified the input file path, enumerated the expected output files, and included imperative directives. This applied equally to the Builder tests (Devstral, Gemma 4, Ornith) and the Explorer tests (Qwen3.6-35B-A3B, Ornith), where the prompt additionally specified MCP tool names (browser_navigate, browser_snapshot, browser_click, browser_type) because the model could not infer them from context alone. This difference in prompt requirements degrades developer experience and increases the overhead of integrating local models into existing workflows. The short-form invocation pattern that makes agentic frameworks practical for daily use does not transfer to local models without additional scaffolding.

Tooling ecosystem instability. During the experiment, Roo Code — the initial VS Code coding agent extension selected for its multi-agent orchestrator mode and Ollama integration — announced its final release and end-of-life, shifting focus to a cloud platform called Roomote. The migration to Cline (the original project from which Roo Code had forked) was straightforward because both extensions use identical tool interfaces, but the incident illustrates a broader risk: the local LLM tooling ecosystem is younger and less stable than cloud AI tooling.

Organizations planning local LLM adoption should evaluate tooling longevity and community health alongside model quality.

Input quality sensitivity. The enriched.md input file used in initial testing contained LOCATOR annotation tags rather than the full ELEMENT annotation format specified in the Builder instructions. Gemma 4 detected this discrepancy and asked a clarifying question; Devstral proceeded silently and generated incorrect login locators as a result. This divergence highlights that local models are more sensitive to input format consistency than Claude, which handles format variations more gracefully. The Ornith Run 2 result adds an important qualification: input sensitivity is not simply a matter of input quality but of input complexity. Supplying Ornith with richer, DOM-verified input degraded its structural decisions rather than improving them. Strict input validation before the Builder stage remains advisable, but validation alone does not guarantee that better input yields better output.

5.6 Training Methodology Versus Parameter Count

The original three-model cohort supported an intuitive conclusion: quality correlates with model size, and the path to closing the local-cloud gap runs through larger models on larger GPUs. The Ornith-1.0-9B result does not support that conclusion.

Model	Parameters	VRAM	Builder Score
Gemma 4 26B-A4B	26B (4B active)	~8 GB active	19/35 (54%)
Devstral Small 2	24B dense	~14 GB	20/35 (57%)
Ornith-1.0-9B	9B dense	~5–6 GB	26/35 (74%)

The smallest model, using the least VRAM, produced the highest score — by a margin of 17 percentage points over a model nearly three times its size. The ordering is inverted relative to parameter count across all three data points, not merely at the extremes.

The most plausible explanation is Ornith's self-scaffolding training methodology, which jointly optimizes code generation and the orchestration framework guiding that generation via reinforcement learning. A model trained to construct and follow its own internal scaffolding for a task is, in effect, trained on the exact capability that the Architecture–Convention Gap measures: holding a large set of explicit procedural rules active while generating structured output. The convention compliance table in Section 4.4.2 shows this directly — the rules Ornith satisfied are precisely the ones that must be recalled rather than inferred.

The practical implication for procurement is significant and runs counter to the recommendation that would follow from the original cohort alone. Model selection criteria should weight training methodology and instruction-following benchmarks above parameter count and VRAM footprint. A 9B model with the right post-training may outperform a 27B model without it, at a third of the hardware cost. Section 7.1 revises the scaling projections accordingly.

5.7 Instruction Delivery Architecture as an Independent Variable

The Ornith Explorer results isolate a variable that the original experiment design held constant and therefore could not observe. Run 1 and Run 2 used the same model, the same hardware, the same scenario, and the same MCP configuration. The only difference was how instructions reached the model: flat agent wrapper files in Run 1, skills-based progressive disclosure in Run 2. The outcomes were 5 of 23 steps with a fabricated technical excuse, versus 24 of 24 steps with zero hallucinations.

This is not a marginal effect. It is the difference between an agent that fails and an agent that succeeds, produced entirely by instruction architecture. The mechanism is consistent with the context utilization limitation identified in Section 5.3: progressive disclosure surfaces a smaller, more relevant instruction set at each decision point, reducing the concurrent context the model must hold. A model that cannot sustain attention across a flat 40 KB instruction file may sustain it perfectly well across the 4 KB slice actually relevant to the current step.

The finding reframes the local LLM adoption question. The quality gap between local and cloud models is not solely attributable to model size, training methodology, or hardware constraints — the way instructions are delivered is an independent variable of comparable magnitude. Organizations adopting local LLMs should invest in instruction architecture optimization alongside model selection and hardware procurement. Practically, this means that a framework whose agent instructions are already structured for progressive disclosure is substantially better positioned for local inference than one relying on monolithic instruction files, regardless of which model is chosen.

> **A Caution on Generalizing This Result**
> Skills-based delivery improved Explorer navigation dramatically, but it did not uniformly improve output. The Ornith Builder Run 2, which also used skills-based instructions, scored lower than Run 1 under flat wrappers — though that run also changed the input file, so the two variables are confounded. What the evidence supports is that instruction delivery materially affects outcomes; it does not establish that progressive disclosure is universally superior for every agent role. Isolating instruction delivery as a single variable across all six agents is identified as future work in Section 8.4.

5.8 Confident Hallucination: A New Risk Category

Ornith-1.0-9B exhibited a failure mode not observed in the previous three models: confident fabrication of technical explanations. Two instances occurred, in different agent roles.

In the Explorer test under flat wrappers, the model stopped after five steps and reported that SauceDemo uses Shadow DOM and Web Components for product cards, making the “Add to Cart” buttons inaccessible. The claim is entirely false — SauceDemo uses plain HTML div elements with standard CSS classes and data-test attributes — but it was delivered with

detailed supporting observations and the technical register of a legitimate finding. In the Builder Run 2, the model generated filter-configuration-modal.locators.json containing locators for date range filters, BOL numbers, hauler checkboxes, invoice numbers, and truck numbers: a coherent, well-formed locator file for a logistics application that has no relationship to the system under test, apparently surfaced from training data.

The distinguishing characteristic of both failures is plausibility. Devstral's wrong selectors, Gemma 4's folder typo, and Qwen's incomplete output are all self-evidently wrong on inspection — a test fails, a path is misspelled, a file is truncated. A fabricated Shadow DOM explanation is not self-evidently wrong; verifying it requires independently inspecting the application's DOM. A well-formed locator file for the wrong application will pass schema validation and may survive casual review.

In an enterprise context this is more dangerous than an honest failure. A model that reports "I could not complete this" prompts investigation. A model that reports "I could not complete this because the application uses Shadow DOM" prompts a design decision — potentially the abandonment of a viable automation approach on false grounds. The risk scales with the reader's trust in the output and inversely with their willingness to verify it.

This elevates the role of the cloud-based Reviewer from quality gate to hallucination detector, and it strengthens the case against removing that gate as local quality scores improve. A 74% local Builder score is not, on its own, evidence that review can be relaxed — the residual 26% now includes a failure class that is specifically resistant to detection. The compensating controls in Section 6.4 should be read with this in mind: deterministic linting catches convention violations but will not catch a syntactically valid locator file for the wrong application.

6. Recommended Hybrid Architecture

6.1 Measured Per-Agent AI Credit Consumption

To evaluate the financial viability of the hybrid architecture, we measured actual billed AI Credit consumption per agent using GitHub Copilot's usage dashboard for a 42-step, medium-to-complex test scenario run through the full pipeline (Healer was not invoked as the Reviewer verdict was APPROVED).

Agent	AI Credits (Measured)	% of Total	Locally Viable?
Explorer	~300	~55%	Navigation yes, reporting no (see 4.5)
Executor	~80	~15%	No — accuracy critical
Reviewer	~72	~13%	No — quality gate and hallucination detector
Builder	~55	~10%	Yes — tested in this experiment
Enricher	~45	~8%	Not tested

Agent	AI Credits (Measured)	% of Total	Locally Viable?
Total	**~550**	**100%**	

Token optimization was applied to the Explorer and Executor agents. Explorer optimization (progressive skill disclosure, replacing Playwright MCP with Playwright CLI) yielded approximately 12% credit reduction. Executor optimization yielded greater than 50% reduction post-optimization. No specific optimization was applied to the remaining agents.

These measurements reflect the framework's production configuration, which includes built-in token optimization: scripts-for-evidence architecture, progressive skill disclosure, and output-aware token management. The data exposes a fundamental constraint on the hybrid cost argument: the Explorer alone consumes approximately 55% of pipeline credits. The Builder — selected for local evaluation because it generates the pipeline's primary deliverable — accounts for approximately 10% of total credits. At the 54–57% quality level of the original three-model cohort, Builder output required three cloud-based fixes to reach passing quality, and the remediation cost offset the credit savings entirely.

The Ornith results improve this arithmetic without resolving it. At 74% quality and roughly 10 minutes per run, the remediation burden for a local Builder is materially lower than at 54–57%, and the speed penalty narrows from 8–9x to approximately 2x. Against that, the same model produced 60% on a second run under different conditions — a 14-point swing that makes per-run remediation cost unpredictable. A pipeline that saves 55 credits but requires an unforecastable number of correction cycles is not a favourable trade at this scale. The financial case for local offloading still begins in earnest at the 24 GB VRAM tier, where the Executor — at approximately 15% of pipeline credits — may also become viable, but the Ornith result establishes that quality is no longer the sole blocker at 16 GB; consistency is.

6.2 Design Rationale

The experimental results point to a differentiated conclusion: not all agents are equal candidates for local offloading, and the boundary now falls within agent roles rather than cleanly between them. The Builder agent was selected for local evaluation because it generates the framework's primary deliverable — page objects, locator JSONs, test data files, and test specifications — the automation code that is the entire purpose of the pipeline. It requires no MCP integration and produces deterministic file artifacts that can be independently verified against the 9-dimension scorecard. The Enrichment Agent, which also operates without MCP and consumes the lowest credits (~45), is a candidate for future local evaluation, though its output (structured scenario files) is an intermediate artifact rather than the pipeline's end product.

The Explorer recommendation requires revision from earlier versions of this analysis. The original conclusion — that the Explorer must remain on cloud inference — was drawn from the Qwen3.6-35B-A3B result and held that MCP-driven navigation was beyond local model

capability. Section 4.5 disproves the general form of that claim: Ornith-1.0-9B completed all 24 navigation steps in approximately 10 minutes, captured 21 correctly-schemad ELEMENT annotations from the live DOM, and produced zero hallucinations when instructions were delivered through progressive skill disclosure.

The revised position separates the Explorer's two responsibilities. Its navigation capability — driving the browser, capturing real selectors, observing application behaviour — is locally viable today with the right instruction architecture. Its reporting compliance is not: inline ELEMENT annotation placement, LEDGER marker emission, and correct output file placement all failed, and these are not cosmetic. Downstream framework scripts parse those markers, and explorer-post-check.js reported 0/24 steps verified against work that was in fact complete and correct. An agent whose output cannot be mechanically verified breaks the scripts-for-evidence architecture described in Section 3.3. Until output formatting compliance reaches the level downstream tooling requires, the Explorer remains a cloud agent — but the reason is now reporting compliance, not navigation capability, and that is a substantially narrower gap to close.

The Executor, Reviewer, and Healer agents remain on cloud because their accuracy requirements justify the cost. The Reviewer's case is strengthened by the Ornith findings: as documented in Section 5.8, it now serves as a hallucination detector as well as a quality gate, and that function becomes more rather than less important as local model scores rise.

6.3 Proposed Architecture

The following architecture is proposed as a target for the 24 GB+ VRAM hardware tier, where projected quality scores and improved run-to-run consistency would make the hybrid model financially viable. It is not recommended for deployment at the 16 GB tier tested in this experiment.

Agent	Deployment	Model	Rationale
Explorer	Cloud (navigation portion locally viable)	Claude Sonnet	Navigation demonstrated locally at 24/24; reporting compliance not yet met
Builder	Local	Ornith-class (instruction-optimized)	Only fully locally viable agent; output verifiable via scorecard
Executor	Cloud	Claude Sonnet	High input tokens, accuracy-critical runtime fixes
Reviewer	Cloud	Claude Haiku	Quality gate and hallucination detector — must not be bypassed
Healer	Cloud	Claude Sonnet	Diagnostic reasoning requires strongest model

Note the change in the Builder model recommendation relative to earlier analysis. Selection should favour models with demonstrated instruction-following characteristics — Ornith-1.0-9B

and successors employing comparable post-training — over larger models with stronger raw coding benchmarks, per the analysis in Section 5.6.

6.4 Compensating Controls for Local Builder

To bridge the Architecture–Convention Gap for the locally-deployed Builder, four compensating controls are recommended. The first three become relevant at the projected 68–80% quality tier, where local output is close enough to production quality that targeted corrections — rather than full remediation — can close the remaining gap. The fourth is a direct response to the hallucination risk documented in Section 5.8 and applies at any quality tier.

First, a post-generation linting pass that programmatically checks for test.step() wrappers, wait strategy presence, credential externalization, and test data externalization — these are deterministic checks that require zero LLM tokens. Second, a template injection mechanism that pre-populates generated spec files with test.step() scaffolding and process.env references, reducing the conventions the model must independently recall. Third, maintaining the cloud-based Reviewer as a mandatory quality gate with no bypass, ensuring that convention violations caught by the scorecard are always flagged for remediation.

Fourth, and newly recommended: an artifact provenance check that validates every generated file against the scenario manifest. The hallucinated filter-configuration-modal.locators.json passed schema validation and would survive a conventional linting pass — it was well-formed, just entirely unrelated to the system under test. A deterministic check that every generated locator file corresponds to a page named in the enriched scenario would have caught it at zero token cost. Organizations deploying local Builders should treat file-set validation as a first-class control rather than an edge case.

7. Scaling Projections

7.1 Hardware Tier Analysis

Earlier versions of this analysis projected quality as an approximately linear function of GPU VRAM and model size within the tested range. The Ornith-1.0-9B result requires that projection to be caveated substantially. A 9B model on 5–6 GB of VRAM scored 74% — above the 68% floor this table projects for a 27B model on a 24 GB GPU, and achieved on hardware from the tier below. Parameter count and VRAM set an upper bound on what a model can hold in context; they do not determine how well it follows instructions within that bound.

The projections below should therefore be read as capability ceilings for models of average instruction-following quality, not as forecasts for any specific model. A well-post-trained model may exceed its tier; a poorly-post-trained one will fall short of it.

Setup	Est. Score (typical)	Gap vs. Claude	Investment
Current tier (24–26B on 16 GB GPU)	19–20/35 (54–57%)	Large	Consumer desktop (~$500 GPU)
Instruction-optimized 9B on 16 GB GPU	26/35 (74%) measured	Moderate	Same hardware, better model
Qwen3.6-27B on 24 GB GPU	24–28/35 (68–80%)	Moderate	RTX 4090 (~$1,600 GPU)
Devstral 2 123B on 48 GB GPU	28–32/35 (80–91%)	Small	Dual GPU or A6000 (~$4,500+)
Claude Sonnet 4.6 (cloud)	32–38/40 (90–95%)	Baseline	Per-token API cost

The second row is the consequential addition. It represents measured rather than projected performance, achieved on the same hardware as the first row, and it indicates that the fastest available route to higher local quality at present is model selection rather than hardware upgrade. This does not eliminate the case for more VRAM — headroom remains necessary for larger context windows and future model sizes — but it reorders the priorities. An organization choosing between a GPU upgrade and a model evaluation cycle should run the evaluation first.

One caveat qualifies the second row heavily: 74% was the better of two runs, with the other at 60%. Neither figure clears the 80% APPROVED threshold, and the spread between them exceeds the gap between the first and third rows. Consistency, not peak capability, is what the projections above do not capture and what production deployment most requires.

7.2 Enterprise Hardware Recommendation

For GPU selection, we continue to recommend the RTX 4090 (24 GB VRAM) as the minimum configuration for production use, with one revision to the reasoning. The RTX 5060 Ti (16 GB VRAM) was previously characterised as a wasted investment on the grounds that it could not exceed the 54–57% quality tier. The Ornith result disproves that specific claim — 74% was achieved on 16 GB — so the recommendation now rests on headroom rather than a hard quality ceiling: 24 GB supports larger context windows, the 27B-class dense models expected to mature through 2026, and concurrent inference alongside the Playwright execution environment. For a pilot program specifically evaluating instruction-optimized small models, 16 GB is now defensible. We further recommend 64 GB DDR5 RAM, a 12+ core CPU, and 1 TB or more of NVMe SSD storage.

7.3 Hardware Trajectory: The Convergence Timeline

The quality gap observed in this experiment is partly a hardware constraint, though the Ornith results demonstrate it is not exclusively one. NVIDIA's 2026 product announcements reveal a deliberate strategy to bring data-center-class memory to consumer and workstation form factors, creating a clear upgrade path for organizations adopting local LLM inference.

At NVIDIA's Computex 2026 keynote, Jensen Huang unveiled the DGX Station — a desktop tower built around the GB300 Grace Blackwell Ultra chip with 748 GB of unified coherent memory. As stated during the keynote: a 70-billion parameter model at full precision runs on this machine the way a browser tab runs on a laptop. The unified memory architecture eliminates the cross-GPU data copying that degrades multi-GPU inference performance, making trillion-parameter models accessible (at quantized precision) from a single desktop.

More significantly for the enterprise QE community, NVIDIA announced a tiered product ladder designed to bring this capability down to accessible price points:

Tier	Hardware	Memory	Model Capacity	Est. Price	Target User
Consumer Desktop	RTX 4090 / RTX 5090	24–32 GB VRAM	27–45B dense	$1,600–$3,500 (GPU only)	Developer, pilot programs
Consumer Laptop	RTX Spark (Fall 2026)	Up to 128 GB unified	120B+ models	TBD (full laptop)	Developer laptop — convergence point
Prosumer Desktop	DGX Spark	128 GB unified	70B full precision	~$4,000 (full system)	Small team, agent prototyping
Value Alternative	Mac Studio M5 Ultra	64–128 GB unified	70B at Q4–Q8	$5,000–$8,000 (full system)	Best capability-per-dollar
Enterprise Workgroup	DGX Station (GB300)	748 GB unified	70B full, multi-model	~$90,000–$100,000 (full system)	Enterprise, regulated data

The trajectory is clear: within 12–18 months, a QE engineer's standard-issue laptop (RTX Spark) could run 120B+ parameter models that our scaling projections place at 85–91% on the framework's 9-dimension scorecard — approaching cloud model parity. The Ornith result suggests convergence may arrive sooner than the hardware roadmap alone implies, since a 9B model on today's consumer hardware already reached 74%. The binding constraint is shifting from what hardware can hold to how models are trained and how instructions are delivered to them.

For organizations evaluating local LLM adoption today, the recommended entry point remains the RTX 4090 (24 GB) at approximately $1,600 for a pilot program, with a planned upgrade path to RTX Spark or DGX Spark as these platforms ship and stabilize. Organizations already holding 16 GB hardware should not treat it as disqualifying: evaluating instruction-optimized small models on existing hardware is now the higher-yield first step.

7.4 Cost–Quality Tradeoff

At the 54–57% quality level of the original three-model cohort, local Builder deployment is not financially viable. The remediation cost of three cloud-based fixes per run offsets the approximately 55 AI Credits saved, and the 8–9x speed penalty makes the workflow impractical for sustained use. At the 74% level demonstrated by Ornith-1.0-9B on the same hardware, the calculation shifts: remediation drops to targeted corrections, and the speed penalty falls to

approximately 2x. Were that score reliably reproducible, local Builder deployment would be defensible at 16 GB today.

It is not reliably reproducible, and that is the operative constraint. The same model returned 60% on its second run. For a pipeline generating dozens of scenarios, the expected remediation cost is a function of the distribution of outcomes, not the best observed outcome — and a distribution spanning 60–74% with an unknown shape cannot be budgeted against. At the projected 68–80% tier (24 GB GPU), the compensating controls described in Section 6.4 could close the remaining gap for integration into generation workflows with a lighter cloud-based Reviewer pass. At the 80–91% tier, local output quality would be near-indistinguishable from cloud for routine scenarios.

8. Conclusions and Future Work

8.1 The Verdict at the 16 GB Tier

Local models on consumer-grade 16 GB GPUs are not ready for production enterprise agentic QE pipelines. That conclusion survives the four-model experiment, but the reasons behind it have changed substantially, and the change matters more than the verdict.

The original three-model cohort failed on capability. Devstral and Gemma 4 scored 54–57%, missed framework conventions wholesale, and took 40+ minutes per Builder run. The natural inference was that 24–26B models on 16 GB hardware simply lack the capacity for dense instruction compliance, and that closing the gap required bigger models on bigger GPUs.

Ornith-1.0-9B falsifies that inference. A 9B model on 5–6 GB of VRAM scored 74%, satisfied nearly every convention the larger models missed, completed the Builder run in 10 minutes, and became the first local model to complete full 24-step Explorer navigation. Capability at this hardware tier is no longer the binding constraint.

Three constraints remain, and each is a genuine blocker for production use. First, consistency: the same model scored 74% and 60% on two Builder runs, a 14-point spread that makes remediation cost unforecastable. Second, hallucination risk: Ornith fabricated a Shadow DOM explanation that would mislead an engineer reading the output, and generated a well-formed locator file for an entirely unrelated logistics application — failures that are plausible enough to survive casual review. Third, output formatting compliance: the Explorer’s otherwise excellent navigation produced annotations and markers that downstream framework scripts could not parse, breaking the mechanical verification the pipeline depends on.

The financial picture is correspondingly nuanced rather than settled. The Builder accounts for approximately 10% of pipeline AI Credits, so even a flawless local Builder saves modestly. At 74% quality with a 2x speed penalty the trade is defensible; at 60% with unpredictable variance it is not; and the experiment cannot yet say which is the expected case. The honest statement is

that the quality argument against local Builder deployment at 16 GB has largely dissolved, the consistency and safety arguments have not, and the financial argument remains modest at this agent's share of the pipeline.

8.2 Value Beyond Immediate Cost Savings

The experiment is not a failure — it is a rigorous baseline. It delivers five outcomes that have value independent of immediate cost savings.

First, a repeatable evaluation methodology. The 9-dimension Reviewer scorecard provides an enterprise-grade quality benchmark that is model-agnostic, scenario-reusable, and more demanding than synthetic benchmarks. Its value was demonstrated concretely by the Ornith test: a model released after the original experiment concluded was evaluated against identical criteria and placed precisely within the existing results, with no methodology changes required.

Second, a precise diagnosis of the quality gap — and its partial resolution. The gap was correctly attributed to instruction-following depth and consistency rather than raw coding ability. Ornith-1.0-9B validates that diagnosis from the opposite direction: a model explicitly post-trained to internalize instruction sets closed most of the gap at a third the parameter count. The diagnosis told model developers what to optimize, and a model optimizing for it performed as predicted.

Third, identification of instruction delivery as an independent variable. The same model, same hardware, and same scenario produced 5/23 steps under flat instruction files and 24/24 under progressive skill disclosure. This is actionable for any organization running agentic workflows regardless of whether they adopt local models, and it means that frameworks already structured for progressive disclosure carry a durable advantage in local deployment readiness.

Fourth, a revised hardware viability threshold. The 24 GB VRAM tier remains the recommended production entry point, but for headroom rather than as a quality floor. The higher-yield first move for organizations holding 16 GB hardware is a model evaluation cycle, not a GPU purchase — an inversion of the guidance that the original three-model cohort would have supported.

Fifth, agent adapter layer portability. The .clinerules and agents/cline/ wrapper architecture demonstrates that the framework's agent instructions can be ported across runtimes (Claude Code, Cline, GitHub Copilot) with minimal changes. This runtime portability is valuable regardless of whether the target is a local model or a cheaper cloud provider.

8.3 Limitations

This study has several limitations that should be considered when interpreting results. All quality scoring experiments used a single scenario (SauceDemo checkout-flow); results may vary for API-heavy, mobile, or data-intensive scenarios. Per-agent AI Credit consumption data was measured on a separate 42-step medium-to-complex scenario, so direct quality-to-cost

correlation within a single scenario was not established. The Qwen3.6-27B dense model could not be tested due to GGUF/Ollama incompatibility. Scaling projections beyond the tested hardware are extrapolations from benchmark scores, not empirical measurements.

Three limitations specific to the Ornith findings warrant emphasis, because those findings drive the most substantial revisions in this paper. First, the two Ornith Builder runs differed in both input file and instruction delivery method simultaneously, so the 74%-to-60% regression cannot be attributed to either variable individually. Second, run-to-run consistency was characterised from two runs per configuration — sufficient to establish that variance exists and is large, insufficient to characterise its distribution. Third, the Explorer instruction-delivery comparison was performed on one model in one agent role; the finding is strong within that scope but has not been replicated across models or across other agents.

8.4 Recommendation and Future Work

The recommendation is not to deploy the hybrid architecture today, but to invest in quarterly re-evaluation using the scorecard baselines established in this paper. Each quarter, run the same standardized scenario through any newly released local model and compare the scorecard against the established baselines. The Reviewer scorecard measures what matters for enterprise deployment: framework convention compliance, not abstract coding ability. When a local model consistently scores above 80% on the 9-dimension scorecard with no dimension below 3 — with emphasis on consistently, across repeated runs — it is ready for production integration.

Four directions warrant investigation before that threshold is reached. First, run-to-run consistency measurement: executing the same model and configuration five to ten times to characterise the distribution of scores rather than sampling it. Given that variance is now the primary blocker, this is the highest-priority open question. Second, isolating instruction delivery method as a single variable across all six agents, holding input constant — the Explorer result is compelling enough to warrant systematic testing of whether progressive disclosure benefits every role or only MCP-intensive ones. Third, developing deterministic artifact provenance validation as described in Section 6.4, since conventional linting demonstrably will not catch well-formed output for the wrong application. Fourth, testing the Enrichment Agent locally, as it shares the Builder’s favourable characteristics — no MCP dependency, structured output, lowest credit consumption — and may provide an additional offload candidate.

The broader conclusion is that the gap between local and cloud models for enterprise agentic QE is closing faster than model size scaling alone would predict, and along a different axis than expected. Training methodology and instruction architecture are proving more consequential than parameter count. Organizations that build evaluation capability now — a scorecard, a standard scenario, a quarterly cadence — will recognise the crossing point when it arrives, and will be positioned to act on it while others are still waiting for larger GPUs.

About the Author

Srinidhi Sreevatsa is a QE Consultant & Architect, Bengaluru. With 20+ years of enterprise quality engineering experience, he specializes in QE transformation engagements, QE maturity assessments, and agentic QE platform development. He is the architect of the Agentic QE Framework v2, the Smart Test Designer RAG pipeline, the 9-dimension quality assessment methodology described in this paper, and Selenium-Playwright Migration kit. He can be reached at srinidhi.ts@gmail.com and linkedin.com/in/srinidhi-sreevatsa-tnarasipura.

— End of Document —